SECURITY AND TRUST

Signing Keys by Proxy

Buy applications were built for one document format at a time: SAP spreadsheets, Adobe Experience Cloud and AEM pages, browser-managed data. Extending to AI and robots reaches households, warehouses, agents and peer mandates. **Rules have to run as memory-safe Rust code with securely random keys, not fail at a document-type rule.** Security counts the most where humans matter the most: in our households.

Every signature answers two questions: *who issued the key*, and *who granted the signer the right to use it*. When a consultancy, a plugin or a platform signs on your behalf with a certificate authority (CA) you never chose, you inherit that CA by proxy: its trust, its revocation service, its jurisdiction, and its failures. October 2026 showed what that costs when the CA is distrusted. This article sets out when you use a CA by proxy, what locks out when its chain breaks, how ECC/SAP signing fits, what each market requires, and how to build a Globalization MVP security first, with a cutover checklist to get there.

What are signing keys, and where are they used?

A signing key is a business's digital seal. It comes in two halves. The **private key** stays with whoever signs, like the stamp locked in a notary's drawer. The **public key** is shared with everyone, so anyone can check that a stamp is genuine without asking the signer. A valid signature proves two things: **who** signed, and that **nothing changed** after they signed. Change one character of a signed contract, firmware file or code release and the check fails.

You already rely on signing keys every day, usually without seeing them:

WHERE	WHAT IS SIGNED, BY WHOM	WHAT IT PROVES	IF THE KEY FAILS
Code on GitHub	Commits and pushes, with a developer's RSA or Ed25519 key	Who made the change, and that it wasn't altered	Anyone with the key can push as that person: revoke it and issue a new one
Firmware and SBOMs	The firmware image and its parts list (SBOM), by the manufacturer	The device runs the real update, and the parts list matches what shipped; the EU Cyber Resilience Act expects this	Devices reject updates, or accept a fake one
E-signatures on legal documents	Contracts, SOWs and filings, with a certificate from a certificate authority (CA)	This person agreed to this exact document, at this time	Signed documents show as invalid if the CA is distrusted
QA attestations in Agile	Review approvals, test runs and release builds, by reviewers and the CI pipeline	The build was reviewed and tested, and no gate was skipped	No proof a release passed QA: an audit gap

The business question is never only "is it signed?" It is also: *whose key, issued by whom, and who granted them the right to sign for us?* A developer's key proves who pushed, but not whether they were allowed to push to that repository. A consultant's certificate proves who signed, but not whether they could sign for your company. That gap between **issuer** and **grant** is what this article is about, and it grows when the key belongs to someone else: a vendor, a plugin, or a consultancy signing **by proxy**.

Why this matters now: keys have to move on events

Historically, signatures lived inside SLA-bound enterprise applications with a heavy price tag, such as SAP and AEM, and governance of those keys was assigned to a consulting team. The keys sat inside the application, and changing who could sign meant a change request, a consulting ticket and a release window. That worked when signing happened a few times a quarter.

It also fitted what those systems were built for. SAP grew from accounting: accumulated ledger rows, in effect spreadsheets at enterprise scale, posted in batches by trusted staff on an internal network. Records like that never needed memory safety, because nothing untrusted reached the code at run time. **Attestation was a timestamp**, the posting date and the change log, not a signature anyone outside could verify. **Data subject request (DSR) evidence was held in pay-to-use data repositories**: vendor add-ons and archives billed by volume or by query, so proving a deletion meant paying to read your own records.

	THEN: SLA APPLICATION	NOW: EVENT-DRIVEN AI
Records	Ledger rows posted in batches by trusted staff	Events from shoppers, partners and agents, arriving all the time
Runtime exposure	Internal network; memory safety not a requirement	Internet-facing edge, where memory-safe code (Rust, Wasm) is the baseline
Attestation	A timestamp and a change log	A signature anyone can verify offline, against a key you publish
DSR evidence	Held in pay-to-use repositories	An append-only ledger you own, with a receipt from every system that acted
Key changes	Change request, consulting ticket, release window	Issued, rotated or revoked on the event

AI work signs on every event: each pull request, each consent change, each new market. Three capabilities stay blocked until the organization can **exchange keys on the event**, issuing, rotating or revoking a key or grant at the moment something happens:

AI CAPABILITY	WHAT IT HAS TO SIGN ON EACH EVENT	BLOCKED WHEN KEYS LIVE IN AN SLA APPLICATION
AI code review	A review attestation per pull request, with a short-lived key the CI pipeline can verify	Keys are held in vendor or consultancy tooling, with no way to issue one per review
Data governance by mandate	A scoped, expiring mandate on every agent action, revoked the moment consent changes	Grants change through tickets and release windows, not on the consent event
AI globalization weighting	Each market's weighting or adapter, published as a signed record when that market's rules or data change	Weights and configuration ship on the application's release cycle, so a market can't be signed for when its rule changes

The risks of leaving keys there. When signing and its governance sit with an external team inside an SLA application, the organization carries risks it doesn't control:

RISK	WHAT HAPPENS	WITH KEY EXCHANGE ON EVENTS YOU OWN
Single point of failure: one developer	The one person who holds a key, or knows how to rotate it, is away or leaves. Releases and revocations stall	Keys live in the organization's key store, not with a person; rotation needs two approvers and follows a written runbook
External supply chain access	A per-seat plugin or application, run by an outside team, holds standing credentials into your systems. Their compromise becomes yours	Each vendor gets a scoped, expiring grant and its own certificate, revocable alone, and every access is a signed record
Knowledge and training data lost at the end	When the subscription or the external team's contract ends, the knowledge base, prompts, labelled data and adapters stay on their platform	The corpus is published as signed records you hold; adapters and training sets sit in your own registry; exit terms require export
Time zone gap	An incident starts while the external team is offline, and nobody on your side can rotate or revoke	Revocation is automated on the event, and your own on-call staff hold pre-approved emergency mandates
Crisis mitigation on breach	During a breach, nobody can say who holds which key, revoke it quickly, or show regulators what happened when. GDPR allows 72 hours to notify	One switch per key and per grant; revocation reaches every system on the event; the ledger's timestamps are the evidence for the notification
Plugins without memory safety, with hard-coded upserts	A plugin or application writes straight into the database with hard-coded upserts, outside the event path: no signature, no consent check, no ledger entry. A memory bug in it can expose keys held in the same process	Plugins hold no signing keys and no direct write access; every write passes the gate as a signed event; the gate runs memory-safe code (Rust, Wasm)
Unsigned device commands and soft firmware: households	Home appliances accept commands from anyone who reaches their login page, as the supermarket controllers did in October 2026, and stream usage data that shows when the home is empty	An encrypted streaming gate the household controls, a certificate per appliance with household-held keys, signed commands within firmware limits, and hardened, signed firmware

Distributed keys and rolling data gating. The answer to these risks is to spread keys across the systems that act, each with its own scoped grant, and to have AI check every write against consent, market rules and mandates as it arrives: a gate that rolls with the data instead of a batch review after it. That model only holds if nothing writes around the gate. A plugin that hard-codes upserts into the data store, or runs in a runtime without memory safety, is a side door: its writes carry no signature, and a single memory-corruption bug can hand an attacker the keys and the write path together. October 2026 made the point twice: php2shell gave unauthenticated code execution in the PHP runtime under every WordPress, Drupal, Magento, Joomla and Laravel host, whatever the plugin code, and an AI gateway package was compromised for the second time this year, again stealing signing keys and AI provider keys from the environment it ran in.

Key exchange on events is what moves signing out of the SLA application and into infrastructure the organization owns. The rest of this article shows what that takes: whose keys, which grants, and where each market's rules apply.

Does the client use eMudhra by proxy?

When a consultancy signs with an eMudhra certificate, it depends on whose certificate signs and who runs the system.

THREE PARTIES

Client — RELYING PARTY
Hires the consultancy
Opens signed PDFs; may have its own CA

hires →
← signed PDF

Consultancy — E.G. BIG FOUR OR IT SERVICES
Signs its own documents
May run signing or systems for the client

certificate from →

eMudhra — CERTIFICATE AUTHORITY
Issues signing certificates
Runs revocation checks (OCSP, CRL)

FOUR SITUATIONS, FOUR ANSWERS

Signs its own — NO

Consultancy signs → client validates

An SOW, invoice or sign-off signed with the consultancy's eMudhra certificate.

The client only checks the chain and revocation. No account, key or contract with eMudhra.

Signs via their setup — YES, INDIRECTLY

Client signer → consultancy → eMudhra

Client people sign through the consultancy's Acrobat Sign account or eSign portal.

Signer identity data goes to eMudhra: effectively a subprocessor the client never chose.

Runs client systems — YES, DIRECTLY

eMudhra cert in the client's AEM or Sign

The consultancy installs eMudhra certificates or enables it as the provider in client systems.

It is now the client's configuration; the consultancy only made the choice.

Client has its own CA — NO

Client signs with its own CA

The client only receives eMudhra-signed documents.

It needs to trust eMudhra's root, usually through Adobe's trust list (AATL).

EVEN AS A RELYING PARTY ONLY, THE CLIENT DEPENDS ON EMUDHRA FOR

Revocation checks

Validating a signature calls eMudhra's OCSP responder or CRL. Long-term validation helps only if it was embedded at signing.

Trust in the root

Not hypothetical: 17–20 Oct 2026, Chrome, Mozilla, Apple and Microsoft blocked eight eMudhra intermediates whose private keys were recoverable. Chrome removes emSign ECC Root CA - G3 from 27 Oct.

Legal weight

Valid under Indian law. Whether it counts as qualified in the EU, or elsewhere, depends on the certificate type and jurisdiction.

AI / TRUST / BOUNDARY LOCKOUT: WHAT STOPS WHEN A CHAIN IS DISTRUSTED

AI lockout — AGENTS

Agent → TLS handshake → blocked chain → no call

Agents, AI gateways and MCP tools calling an API whose certificate chains through a blocked intermediate fail the handshake: the tool call never happens.

Mandates signed with keys from an exposed environment must be rejected, and code-signed packages and adapters fail verification.

Trust lockout — RUNTIMES

Browser, OS, Node, Composer, certifi reject the chain

A distrusted chain is not a warning: browsers and runtimes refuse the connection, so the site or API is unreachable.

Signed documents on that chain show as invalid wherever they are opened.

Boundary lockout — CROSSINGS

Edge ↔ origin, webhooks, feed syncs, partner APIs

Every boundary crossing is a TLS handshake. A distrusted chain on either side stops the crossing: feeds stall, webhooks fail.

Consent, opt-out and deletion events stop propagating: a DSR lockout. Queue them and replay once the chain is reissued.

Recover faster: know which certificates chain through which CA, automate reissue (ACME) with a second CA ready, and sign records with keys you own and can verify offline.

WHEN IT IS BY PROXY, THESE BECOME CONCRETE

Vendor and subprocessor lists

eMudhra belongs in the client's third-party register and data processing agreement, through the consultancy's contract.

Personal data

Signer identity checks, including Aadhaar eSign, process personal data in India: residency and transfer questions for EU and UK signers.

Exit

If the consultancy leaves, the signing setup and certificate relationship may go with it.

CONTRACT CLAUSES TO ASK FOR

Disclose every provider

Every trust service provider and CA used for the client's signers or systems.

Certificates issued to the client

Certificates installed in client systems name the client, not the consultancy.

Embed LTV

Revocation info and timestamps embedded so documents stay verifiable after the relationship ends.

CERTIFICATES YOU CONTROL: NO THIRD-PARTY CHAIN TO INHERIT

crm-sync certificate link — OWN KEY

crm-sync.dev/license/verify?cert=…

Licence grants, upload attestations and deletion records are signed with crm-sync's Ed25519 key, published at /.well-known/jwks.json.

Anyone verifies offline, with no account and no CA chain that can be distrusted. Short links at /v/<hash>; print QR at /license/qr/<hash>.svg.

Xano self-host certificate — YOUR CA

Your domain, your TLS certificate, your trust store

Self-hosted Xano runs on infrastructure you operate: you choose the CA for its TLS certificate and keep the private key in your own HSM or key vault, in country.

You also own the trust store: confirm the Node and Composer bundles reject the October 2026 eMudhra chains.

ECC / SAP SIGNING: THE SAME SCRUTINY, PLUS TWO ECC-SPECIFIC RISKS

What gets signed	Mechanism in ECC	Who usually sets it up	Proxy risk
PDF forms Invoices, POs, delivery notes	Interactive Forms by Adobe through Adobe Document Services (ADS), with a .pfx credential	Consultants, during the build	Directly: the CA behind that credential is now the client's
Approvals and records QM, batch records, change master, 21 CFR Part 11	SAP digital signature through SSF, keys in PSE files managed in STRUST	Basis team or consultants	Directly, if they imported keys or certificates from their CA
Outbound HTTPS Shopify, Google, banks, tax portals	ICM web server, SSL client PSEs in STRUST	Basis team or consultants	Trust store: ECC may still trust a chain everything around it now refuses
India statutory filings GST, e-invoice, company filings	Digital Signature Certificates (DSCs) on USB tokens; eMudhra is a large issuer	Often the consultancy or an outside CA firm	Grant problem: the certificate names a consultant, not the client
Spreadsheets Migration templates, LSMW, Migration Cockpit, reports	Usually unsigned; sometimes Office signatures with a personal certificate	Consultants	No signature at all, so no provenance

STRUST doesn't update itself

Browsers, Node, Composer and operating systems shipped the eMudhra distrust in October. An ABAP trust store changes only when someone edits it.

Check every PSE in STRUST, plus ADS and Java stack keystores.

A certificate is not authority

A CA proves who holds the certificate, not what they may sign. A consultant's DSC on a client filing needs a written grant: who, what, how long, revoked on exit.

The S/4HANA move carries trust

A system conversion (SUM with DMO) migrates the PSEs too. Clean out stale or proxy certificates first; ECC mainstream maintenance ends in 2027.

Unsigned spreadsheets

Migration data handed over by consultants has no issuer. Hash each handoff file and record it, signed with a key you own: the same ledger as firmware.

Checklist for the consulting engagement

- Export every PSE and certificate in STRUST, with its issuer chain and who imported it
- Name the ADS signing credential: issued to whom, by which CA
- List every DSC used for statutory filings, who holds the token, and the grant behind it
- Require certificates in client systems to name the client
- Hash and log every spreadsheet and PDF handed over

RULES MANAGEMENT: IN-COUNTRY DATA AND TRANSFER, WITH CLOUDFLARE AND XANO

Country	Law	Data in country	Cross-border transfer	Cloudflare	Xano
China	PIPL, Data Security Law, Cybersecurity Law	Required for critical infrastructure operators and above volume thresholds; important data stays	Security assessment, standard contract or certification, unless an exemption applies	China Network via JD Cloud (Enterprise), needs an ICP filing; Geo Key Manager picks key regions	No mainland region: self-host on a mainland cloud, with its own TLS certificate from a CA you choose
Japan	APPI	Not required by law	Consent naming the country, or a recipient with equivalent safeguards; EU adequacy	Regional Services and Geo Key Manager keep TLS processing and keys in Japan	A Japan-region instance where offered, or self-host in Japan
Korea	PIPA (2023 amendment)	No general rule; sector rules for finance and public-sector cloud	Consent before transfer, or a certified or adequate recipient; disclose country and recipient	Regional Services and Geo Key Manager; check the Korea region	A Seoul-region instance, or self-host in Korea
Vietnam	PDP Law (from 2026), Decree 53 (Cyber Law)	Some services must store in Vietnam; foreign firms when ordered by the regulator	Transfer impact assessment filed with the Ministry of Public Security	No local residency product; keep personal data out of edge logs and caches	Self-host in Vietnam where localisation applies; otherwise a nearby region with the filing
Singapore	PDPA	Not required by law	Comparable protection: contract, binding corporate rules or certification	Regional Services and Geo Key Manager; check the region list	A Singapore-region instance where offered
Taiwan	PDPA (2023 amendment)	Not general; sector rules	Allowed unless the regulator restricts a destination, e.g. mainland China for some sectors	Standard network; keep keys out of restricted regions	Nearest region or self-host; never route through a restricted destination
EU · Germany	GDPR, BDSG, TDDDG (cookies)	Not required by law; often a contract term for public sector and health	Adequacy (incl. EU-US DPF, Japan, Korea, UK) or SCCs with a transfer assessment	Data Localization Suite: Regional Services, Customer Metadata Boundary, Geo Key Manager (EU)	An EU-region instance, or self-host in Germany
UK	UK GDPR, DPA 2018, Data (Use and Access) Act 2025	Not required by law	UK adequacy regulations, IDTA or Addendum; UK-US data bridge	Regional Services; check the UK region	A UK or EU-region instance, or self-host in the UK

A signature you only validate is a dependency. A signing setup your people use, or a certificate in your systems, is a vendor.

Patterns, not a claim about any firm. eMudhra incident per The Hacker News and Help Net Security, October 2026. Country rules summarised; check Cloudflare and Xano region lists. Not legal advice.

Yoonsun Lee · YSL150 LLC

1. When you use a CA by proxy

Three parties are involved: the **client** (you), the **consultancy** or vendor, and the **CA** that issued the consultancy's certificate. Whether you use that CA depends on whose certificate signs and who runs the system.

SITUATION	DO YOU USE THE CA?	WHY
The consultancy signs its own documents (SOW, invoice, sign-off)	**No**	You are a relying party: you validate the chain and revocation. No account, key or contract with the CA
Your people sign through the consultancy's setup (its Acrobat Sign account or eSign portal)	**Yes, indirectly**	Your signers' identity data goes to the CA: effectively a subprocessor you never chose
The consultancy builds or runs your systems and installs the CA's certificates in your AEM, Acrobat Sign or SAP	**Yes, directly**	It is now your configuration; the consultancy only made the choice
You sign with your own CA and only receive the consultancy's signed documents	**No**	You need to trust the CA's root, usually through Adobe's trust list (AATL) or the OS store

Even as a relying party only, you depend on the CA for **revocation checks** (its OCSP responder or CRL; long-term validation helps only if it was embedded at signing), **trust in its root**, and the **legal weight** of the signature in your jurisdiction.

These are patterns, not claims about any firm. A contract, or the certificate chain of a signed PDF, shows which provider a given consultancy uses.

2. Why it is not hypothetical: October 2026

A key ceremony at eMudhra on 30 April 2025 generated eight intermediate CA keys with too little randomness. Between 17 and 20 October 2026 a researcher recovered all eight private keys from the public certificates, and Chrome, Mozilla, Apple and Microsoft blocked them: six TLS intermediates and two

code-signing intermediates. Chrome removes the emSign ECC Root CA - G3 from devices from 27 October. Go, NSS, Ubuntu, Debian and Composer shipped trust-store fixes on 20 October, and the Node.js releases of 21 October also reject the chains. India's Digital Signature Certificates run under India's own CA licensing rather than the browser root programs and, as reported, were not directly affected.

The keys were not stolen; they were computable. An audited process with certified hardware still produced weak keys, so the lesson is about provenance: know which of your certificates chain through which CA, and sign your own records with keys you own and can retire. Background: Hydrogen or bespoke stack, Org Trust Layer, and reporting by The Hacker News and Help Net Security.

The same month, on the shop floor: refrigeration controllers in France and the UK

The eMudhra keys were weak; in October's other incident there were no keys at all. On the night of 11 October 2026, attackers reached the refrigeration controllers of hundreds of French supermarkets, including Intermarché, Netto, Coopérative U, Carrefour, Monoprix and E.Leclerc stores. The Danfoss controllers ran web servers open to the internet, and many accepted the username `admin` with the current date as the password. The attackers set cases to defrost with food inside: some reached about 50 °C within six hours, some equipment melted or caught fire, and the change went unnoticed because it raised no alarm and no staff were present. No data was stolen; the losses ran to tens of millions of euros. Danfoss had shipped a fix for its current AK-SM800A series in August 2025, but older units were end-of-life. The Prime Minister asked ANSSI to investigate, and its director said the controllers urgently needed disconnecting from the internet.

On 13 October, UK Cyber Defence found more than 800 UK refrigeration and building controllers with their login pages on the open internet. By early

November, UK victims were confirmed: a West London wholesaler, Langridge Organic, found six of its ten cold-room controllers, running Copeland's Dixell XWEB software, set to defrost with a 150 °F maximum and their temperature alerts switched off. Manual checks saved its stock. The NCSC said it was aware of a small number of affected UK organisations. On 2 November, similar attacks hit supermarket refrigeration in Belgium, Luxembourg, the Netherlands, Sweden, Estonia, Ireland and Portugal. No one has been publicly identified as responsible.

Both incidents are the same signing-keys problem. A controller that accepts any command from anyone who reaches its login page has no **issuer** (no device or operator certificate) and no **grant** (nothing says who may change a setpoint, by how much, or when). The attacker also switched off the alarm through the same channel, so the record of what changed was lost with it. Sources: Wikipedia, Public Sénat, and The Grocer on UK exposure and UK victims.

Households: where humans matter the most

Security counts the most where humans matter the most: in our households. The October attacks hit shops, but the same class of device sits in homes: fridges and freezers, heat pumps, boilers, thermostats and EV chargers, all on the household network and most talking to a manufacturer's cloud. A home has no security team and no morning temperature check. If a supermarket controller can be set to defrost from the internet, a home appliance with an open login page or a default password can be too, and its data stream says far more about the household: when the fridge door opens, when the heating comes on, when nobody is home.

Why households wire them up. The benefits are real, and they are why the answer is to secure the stream, not to unplug:

USE	BENEFIT TO THE HOUSEHOLD
Heating, heat pumps and thermostats	Lower bills: heating runs on schedules and on cheaper time-of-use tariffs, and only in rooms in use
Fridges and freezers	Less food waste: a warning when the door is left open or the temperature drifts
Cameras, doorbells and baby monitors	Watching over children, pets and deliveries while away
Sensors for older relatives	Living independently for longer, with an alert to family if something changes
EV chargers and home batteries	Charging when power is cheapest and cleanest
Printers and scanners	Printing from any phone or laptop in the home, scanning straight to email or cloud storage, and ink or toner reordered before it runs out
Loyalty programmes	Member prices, points and offers that match what the household actually buys
The TV and phone together	One screen and one remote for the whole home

The TV and the phone have become the home's micro-network. Many households now run the home from a phone on the sofa and a smart TV on the wall: casting to the screen, using the phone as the remote, and letting the TV act as the hub for lights, cameras, speakers and appliances. Newer devices join a local mesh (Thread) and a common smart-home standard (Matter), so the phone, the TV and each device talk inside the home, without a round trip to every maker's cloud. That is good for speed, for working when the internet is down, and for privacy.

It also makes the TV and the phone the **gate**. Whatever reaches them can reach every device paired to them, and TVs often run for years without updates while streaming viewing data to their makers. Matter shows the right pattern: every certified device carries its own **device attestation certificate** from its manufacturer, and pairing gives it a fresh **operational certificate** issued by the

household's own network, so the household, not the maker, holds the keys that let a device join or leave. The household's gate should work the same way for everything else: local first, encrypted, and under keys the household holds.

Cameras show what is at stake. In late November 2025, Korean police arrested four people over the hacking of about **120,000 internet-connected cameras** in homes and businesses, including private homes, a Pilates studio and a gynecology clinic. Two of them accounted for most of the hacks and made sexually exploitative videos from the footage, which were sold on an overseas site; police also arrested buyers and moved to close the site. The way in was simple: the cameras had weak passwords, such as repeated characters or sequential numbers and letters. Many had been installed to watch over children and pets. A leaked video can't be recalled the way a spoiled freezer can be restocked, which is why the stream itself has to be gated. Sources: BleepingComputer, Kaspersky, The Star.

Households need two things before anything else: **an encrypted data streaming gate**, and **hardened firmware**.

REQUIREMENT	WHAT IT MEANS AT HOME	WHAT IT STOPS
No open login page, no default password	Each appliance ships with a unique credential and no management page on the internet	The October method: find the page, type `admin` and the date
An encrypted data streaming gate	Telemetry and camera video leave the home only encrypted, through one gate the household controls, and only for the purposes the household agreed to	Usage data revealing when the home is empty; silent streaming to parties the household never chose
Keys held by the household	Every appliance has its own certificate, and the household holds the keys that pair it to apps and services, so access can be revoked one device or one service at a time	A manufacturer, installer or app keeping standing access after the household has moved on
Signed commands within safe limits	Remote commands from an app or cloud are signed, checked against what the household granted, and kept within limits set in firmware	A fridge told to run a 50 °C defrost, or a heating system turned off in winter
Hardened firmware	Updates are signed and verified before install, the device boots only signed code, memory-safe components handle network input, and an SBOM lists what is inside	A forged update, a memory-corruption bug becoming remote control, or an unpatched unit nobody knew was vulnerable
A stated support period	The maker says how long security updates will come, and the device warns before they stop	Appliances that keep running years after their last patch, as the end-of-life French controllers did
The TV and phone as the gate	The hub (TV, phone or home hub) gets security updates for as long as it runs the home, appliances sit on their own network segment, and pairing uses the household's own certificates (as Matter does)	An old, unpatched TV becoming the way into every device paired to it
Local alarms and local control	Alerts sound on the device and work without the cloud; the cloud can't switch them off	An attacker silencing the warning while changing the setting

The rules are arriving. The UK's product security regime has banned universal default passwords on connectable consumer products since April 2024 and requires a stated support period. In the EU, the Radio Equipment Directive's cybersecurity requirements have applied to wireless devices since August 2025,

and the Cyber Resilience Act adds vulnerability reporting from September 2026 and full security obligations, including secure updates and an SBOM, from December 2027.

The same controls protect lights-out sites, where cold rooms, chillers and server cooling run with nobody on the floor; see Entitlement Security.

3. AI security: trust and boundary lockout

A distrusted chain is not a warning. Browsers and runtimes refuse the connection, and everything that depended on it stops.

LOCKOUT	WHAT STOPS	SIGNAL
AI lockout	Agents, AI gateways and MCP tools calling an API whose certificate chains through a blocked intermediate fail the TLS handshake, so the tool call never happens. Mandates signed with keys from an exposed environment must be rejected; code-signed packages and adapters fail verification	Denied tool calls, handshake errors in agent logs
Trust lockout	Browsers, operating systems, Node, Composer and Python's `certifi` refuse the chain: the site or API is unreachable. Signed documents on the chain show as invalid wherever they are opened	Unreachable origins, invalid signature banners
Boundary lockout	Every crossing (edge to origin, webhooks, feed syncs, partner APIs) is a TLS handshake. Feeds stall and webhooks fail. Consent, opt-out and deletion events stop propagating: a **DSR lockout**	Stalled feeds, growing webhook retries, unacknowledged deletions

Recover faster: keep an inventory of which certificates chain through which CA, automate reissue (ACME) with a second CA ready, queue DSR events so they can be replayed with their timestamps, and sign records with keys you own so they still verify when a public CA does not.

4. Tools and permissions: a certificate is not a grant

A CA proves who holds a certificate. It does not prove what they may sign or do. That needs a **grant** from the organization: who, what, for how long, revoked on exit. For AI agents the grant is a **mandate**: scoped to tools and markets, expiring, and checked on every action.

UCP agent mandates: the permissions and authorization funnel

In agentic checkout, the mandate is a signed, checkable object. The Universal Commerce Protocol (UCP), co-developed by Shopify and Google, carries an agent from discovery to post-purchase, and plugs into the Agent Payments Protocol (AP2) for payment authorization. AP2 expresses permission as signed mandates: an **intent mandate** (what the shopper asked for, within which limits) and a **cart mandate** (the exact cart the shopper approved), with the payment passed on as a token, never card data. Each stage of the funnel needs three things: an **idempotent ID**, so a retry never acts twice; the **consent** in force at that moment; and an **authorization** the gate can verify.

FUNNEL STAGE	MANDATE OR RECORD	IDEMPOTENT ID	CONSENT	AUTHORIZATION
1 · Intent	Intent mandate: what the shopper wants, a price cap and an expiry, signed by the shopper	Intent ID	Purposes read; no personal data needed yet	The agent may search and compare, nothing more
2 · Cart	Cart mandate: exact items, prices and merchant, approved by the shopper or within the intent's limits	Cart ID, plus a key on every cart update	`checkout.buyer.consent` from UCP's buyer-consent extension: only `source: "platform"` records the buyer's choice; `source: "business"` is a default, not consent	The merchant checks the cart sits inside the intent mandate
3 · Checkout	The UCP checkout session	Checkout ID, plus a key on completion	Missing consent data (say, SMS chosen with no phone number) is a warning, and checkout must not complete	The gate verifies the mandate signatures and the agent's grant
4 · Payment	Payment token to the payment provider	One key per charge, so a retry can't charge twice	Ads-data consent decides whether the conversion goes to Google	The payment provider authorizes; the agent never sees card data
5 · Post-purchase	Order, return or subscription change	Order ID, RMA ID, a key per change	Marketing consent checked at send time	The agent acts only within its mandate: returns and changes as granted
6 · Revoke	A mandate-revoked or consent-withdrawn event	The event's own ID, logged once	Withdrawal reaches every system on the event	The gate refuses any further action under that mandate

DX, AX and UX: who the funnel has to work for. The same mandate has three audiences, and each needs something different:

EXPERIENCE	WHO	WHAT THEY NEED FROM THE FUNNEL
DX (developer experience)	The teams building the store, the gate and the integrations	One schema for mandates and consent; an idempotency key on every state change; signed responses they can verify against a published key set; test mandates in a sandbox; clear failures, such as a missing-consent warning, instead of silent drops
AX (agent experience)	The AI agents shopping on someone's behalf	A machine-readable profile saying which capabilities the store supports; mandates that are scoped and expiring; deterministic answers; retries that are safe because of idempotency; declines that say why, so the agent can ask the shopper rather than guess
UX (user experience)	The shopper, and the household	A plain view of what the agent may do and up to how much; approval once, within limits; revoke at any time; a receipt for each action; consent asked once and remembered, not re-prompted

The funnel is where all three meet: if the developer can't verify a mandate, the agent can't trust the answer, and the shopper can't trust the agent.

Certificates you control

	CRM-SYNC CERTIFICATE LINK	XANO SELF-HOST CERTIFICATE
What it is	Licence grants, upload attestations and deletion records signed with crm-sync's Ed25519 key, published at `/.well-known/jwks.json`	Self-hosted Xano on infrastructure you operate, with a TLS certificate from a CA you choose
How it is checked	`crm-sync.dev/license/verify` : anyone verifies offline, with no account and no CA chain that can be distrusted; short links at `/v/<hash>` , print QR at `/license/qr/<hash>.svg`	Your trust store: confirm the Node and Composer bundles reject the October 2026 eMudhra chains
Key custody	The platform's signing key; the public half is published	Your own hardware security module (HSM) or key vault, in country

Assets that prove provenance (PDFs and firmware, images, 3D files, docs) are vaulted the same way: each file under its own key, every upload certified, every download granted and recorded in a hash-chained ledger. See the

Protected Firmware Asset Management and Download Service and Firmware asset publishing.

Xano release gate: TLS opens the connection, the helmet decides what loads

TLS proves which server is on the other end of the connection. It doesn't prove which theme is calling, or that the shopper behind it consented. That isn't left to each platform's plugin or module. Every theme loads the same way, through the **Cloudflare helmet**: a higher-order function at the edge that wraps the page before anything runs. Webflow to 11ty themes already load this way, and WordPress, AEM, Drupal, Joomla and Shopify themes load the same. It doesn't matter how or when the CMS publishes.

Why it loads the same everywhere: it's vanilla, a core tenet of the Design Rules. The helmet and what it loads are plain HTML, CSS and JavaScript. Petite-vue is the default for behaviour, and HTMX is opt-in. There's no React bundle, no build step tied to a framework and no theme API to integrate with. Any CMS that can output a page can carry it, whether that's a Liquid theme, a PHP template, an AEM component or a Twig template. The theme doesn't need to know the helmet exists.

The helmet runs on every request, in this order:

1. **Consent loads first.** Nothing else on the page runs until consent is read.
2. **Is the surface paired?** The origin must be registered to this tenant, at `/stack/config`.
3. **Caps and mandate.** The caller's capabilities are checked, and any agent action needs a mandate.
4. **Signed release request.** The helmet, not the theme, signs the request to the Xano release gate.

The Xano release gate then checks consent at the system of record, along with the market, caps and residency rules. If any check fails, it refuses the release

and still writes the refusal to the release ledger. A kill switch refuses every release at once.

CONNECTED THEME	WHAT THE CMS PUBLISH OWNS	WHAT LOADS THROUGH THE HELMET (THE SAME FOR EVERY THE
Webflow → 11ty	Pages, CMS items, design	Consent, stack loader (vanilla JS/CSS/HTML, petite-vue or HTMX), caps, mandate check, signed release request
Shopify	Theme, sections, app embeds	The same
WordPress	Theme, blocks, page builder	The same
AEM	Components, templates, Dispatcher cache	The same
Drupal	Theme, modules, views	The same
Joomla	Template, extensions	The same
SAP (back office)	No theme: data through Integration Suite on a 15-minute batch	The same gate for outbound data, with a client certificate on t integration user

Why the CMS doesn't matter. No theme, plugin or module holds the gate secret, so a WordPress plugin update, an AEM component from a consultant or a Joomla extension can't widen what gets released. Publishing changes what a page shows. It never changes what the gate sends, because that decision sits at the edge and the system of record, outside the CMS. A new platform needs one thing: put its origin behind the helmet and pair it.

ECC/SAP signing: the same scrutiny, plus two ECC-specific risks

WHAT GETS SIGNED	MECHANISM IN ECC	WHO USUALLY SETS IT UP	PROXY RISK
PDF forms (invoices, POs, delivery notes)	Interactive Forms by Adobe through Adobe Document Services (ADS), with a `.pfx` credential	Consultants, during the build	Directly: the CA behind that credential is now yours
Approvals and records (QM, batch records, change master, 21 CFR Part 11)	SAP digital signature through SSF, keys in PSE files managed in `STRUST`	Basis team or consultants	Directly, if they imported keys or certificates from their CA
Outbound HTTPS (Shopify, Google, banks, tax portals)	ICM web server, SSL client PSEs in `STRUST`	Basis team or consultants	Trust store: ECC may still trust a chain everything around it now refuses
India statutory filings (GST, e-invoice, company filings)	Digital Signature Certificates on USB tokens	Often the consultancy or an outside CA firm	Grant problem: the certificate names a consultant, not the client
Spreadsheets (migration templates, LSMW, Migration Cockpit, reports)	Usually unsigned; sometimes Office signatures with a personal certificate	Consultants	No signature at all, so no provenance

- `STRUST` **doesn't update itself.** Browsers and runtimes shipped the distrust in October; an ABAP trust store changes only when someone edits it. Check every PSE in `STRUST`, plus ADS and Java stack keystores.
- **The S/4HANA move carries trust.** A system conversion (SUM with DMO) migrates the PSEs too. Clean out stale or proxy certificates first; ECC mainstream maintenance ends in 2027.
- **Unsigned spreadsheets are the weakest link.** Hash each handoff file and record it, signed with a key you own.

Stores: POS registers in sync with warehouse and RMA systems

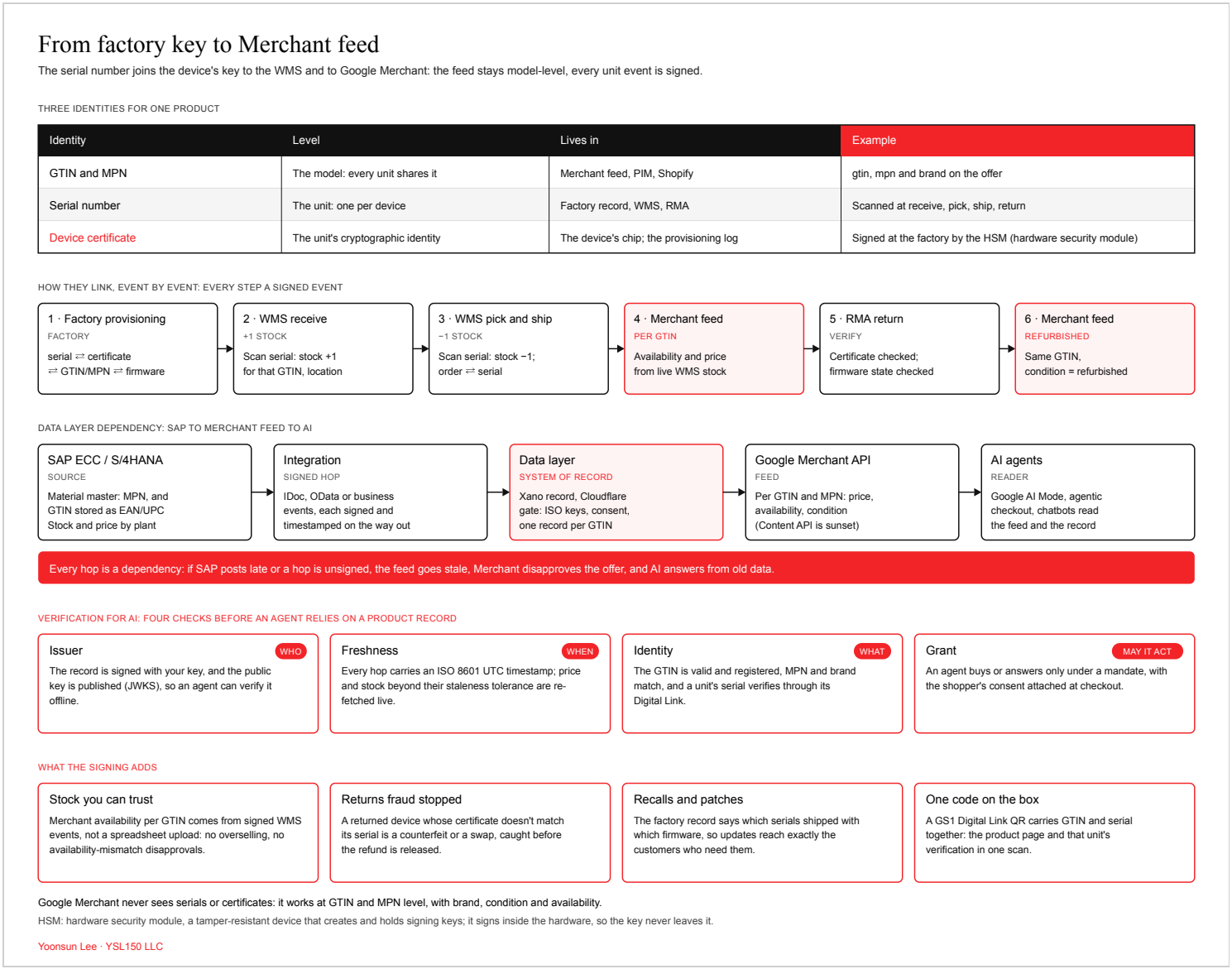
A point-of-sale register is a device that moves money and stock. When registers, the warehouse management system (WMS) and the returns system (RMA, return merchandise authorization) sync by batch file or shared credentials, nobody can say which register recorded a return, who authorised the refund, or whether the item ever reached the warehouse. Keeping them in sync safely means every step is a **signed event** from a known device, under a grant, and each system acts only on events it can verify:

EVENT	SIGNED BY	WHAT THE RECEIVING SYSTEM CHECKS	WITHOUT IT
Sale at the register	The register's device certificate, plus the cashier's session	Price matches the published record; stock is reduced once; payment is a token, never card data	Stock drifts, prices are disputed, and card data enters PCI scope
Return at the counter	The register and cashier, citing the original sale's signed record	The return matches a real sale and is within policy; an RMA number is issued	Refunds on items never sold there: return fraud
RMA authorised	A service agent, or an AI agent under a mandate	Refund limit, market and reason within the grant	Agents, contractors or plugins issuing refunds without authority
Item received at the warehouse	The scanner's device certificate	The item matches the RMA and its condition is graded	A refund paid before the item arrives, or stock counted twice
Refund released	The payment service, only after the receipt record exists	One refund per RMA, enforced by an idempotency key	Double refunds when a sync retries
Store goes offline	The register queues its signed events, each with an ISO 8601 UTC timestamp	Events replay in order; duplicates are dropped by key	Lost sales and returns, and a store that never reconciles with the warehouse

The same rules from the rest of this article apply: a certificate per register and scanner, revocable one at a time; no plugin writing stock or refunds around the gate; and a ledger that shows, for any refund, which register, which person or agent, and which warehouse receipt it rested on.

From factory key to Merchant feed. The serial number is what joins the device's key to the WMS and to Google Merchant. The **GTIN** and **MPN** identify the model, and every unit shares them. The **serial number** identifies the unit. The **device certificate**, signed at the factory by the manufacturer's HSM, proves the unit is genuine. The factory's provisioning record ties all three to the firmware version. WMS events scan the serial at receiving, picking, shipping and returns, so the stock Merchant reads per GTIN comes from signed events, and a returned device whose certificate doesn't match its serial is caught before a refund. Google Merchant never sees serials or certificates: it works at GTIN and MPN level, with brand, condition and availability.

Behind the feed sits a chain of dependencies: SAP holds the material master (the MPN, and the GTIN stored as EAN/UPC) and stock by plant; an integration hop (IDoc, OData or business events) carries it out; the data layer keeps one signed record per GTIN; the Merchant API publishes it; and AI agents, from Google AI Mode to agentic checkout and chatbots, read the result. If SAP posts late or a hop is unsigned, the feed goes stale, Merchant disapproves the offer, and AI answers from old data. Before an agent relies on a product record it should check four things: the **issuer** (signed with your key, verifiable offline against your published JWKS), **freshness** (a UTC timestamp on every hop, with price and stock re-fetched live past their tolerance), **identity** (a valid GTIN, matching MPN and brand, and a serial that verifies through its Digital Link), and the **grant** (a mandate, with the shopper's consent at checkout).



Loyalty feeds and time-sensitive consent. Loyalty adds member prices and points to the same feed, and conversions back to Google. Both depend on consent that changes over time: a shopper can allow ads use today and withdraw it tomorrow. So every step carries the consent state *at the moment of the event*, and personal and card data stay behind the gate:

STEP	WHAT MOVES	THE RULE
Member joins	A pseudonymous loyalty ID, and consent per purpose, channel and market, each with a timestamp	Consent is recorded as an event, not a checkbox stored once
Member price published	Member prices and points per GTIN in the Merchant feed (its loyalty programme attributes) and on site	Published as a signed record; price comes live from the system of record
Purchase	Order value, the loyalty ID and a hashed email	Card data stays tokenised with the payment provider and never enters a feed (PCI); personal data is hashed before it leaves (PII)
Conversion sent to Google	Value and hashed identifiers, with the consent state at the time of purchase	Sent only if ads-data consent was granted at that moment; a later withdrawal stops future sends
AI answers about member prices	Price, points and offers	Read from published records; personalised only for a signed-in member with consent; CRAG declines rather than guesses
Withdrawal or deletion	A consent-change or deletion event	Reaches the feed, conversions and audiences on the event, with a receipt from each

Loyalty and subscription applications need AI memory-safe binding. Loyalty and subscription apps (points, tiers, member prices, renewals, pauses and skips, from apps such as Smile.io, LoyaltyLion and Yotpo for loyalty or Recharge and Skio for subscriptions) hold entitlements that move money: a redemption, a free item, a renewal charge. Once an AI agent can act on them for a member, each action has to be *bound*: tied to the member's identity, their consent at that moment, the payment token and a grant, in one signed record that the gate checks in memory-safe code before anything happens. Binding in the plugin's own database, or in a runtime without memory safety, is the side door this article describes: a hard-coded upsert can grant points or skip a charge with no signature, and a memory bug can expose the payment and loyalty keys together.

ACTION AN AGENT TAKES	BOUND TO	CHECKED AT THE GATE
Redeem points or apply a member price	Member ID, tier, the points balance record and a mandate	The balance is read live, not from memory or embeddings; the redemption is signed once, with an idempotency key, so a retry can't redeem twice
Pause, skip or change a subscription	Member ID, the subscription record and the member's instruction	Only within what the member granted; every change is signed and timestamped
Renew and charge	The payment token (never card data), the renewal date and current consent	The charge runs only if the subscription is active and consent hasn't been withdrawn
Send offers or win-back messages	Marketing consent per channel and market, at send time	No consent at that moment means no send, whatever the segment says
Cancel or delete	A cancellation or deletion event	Reaches the app, the feed and Google conversions on the event; balances are settled, then removed

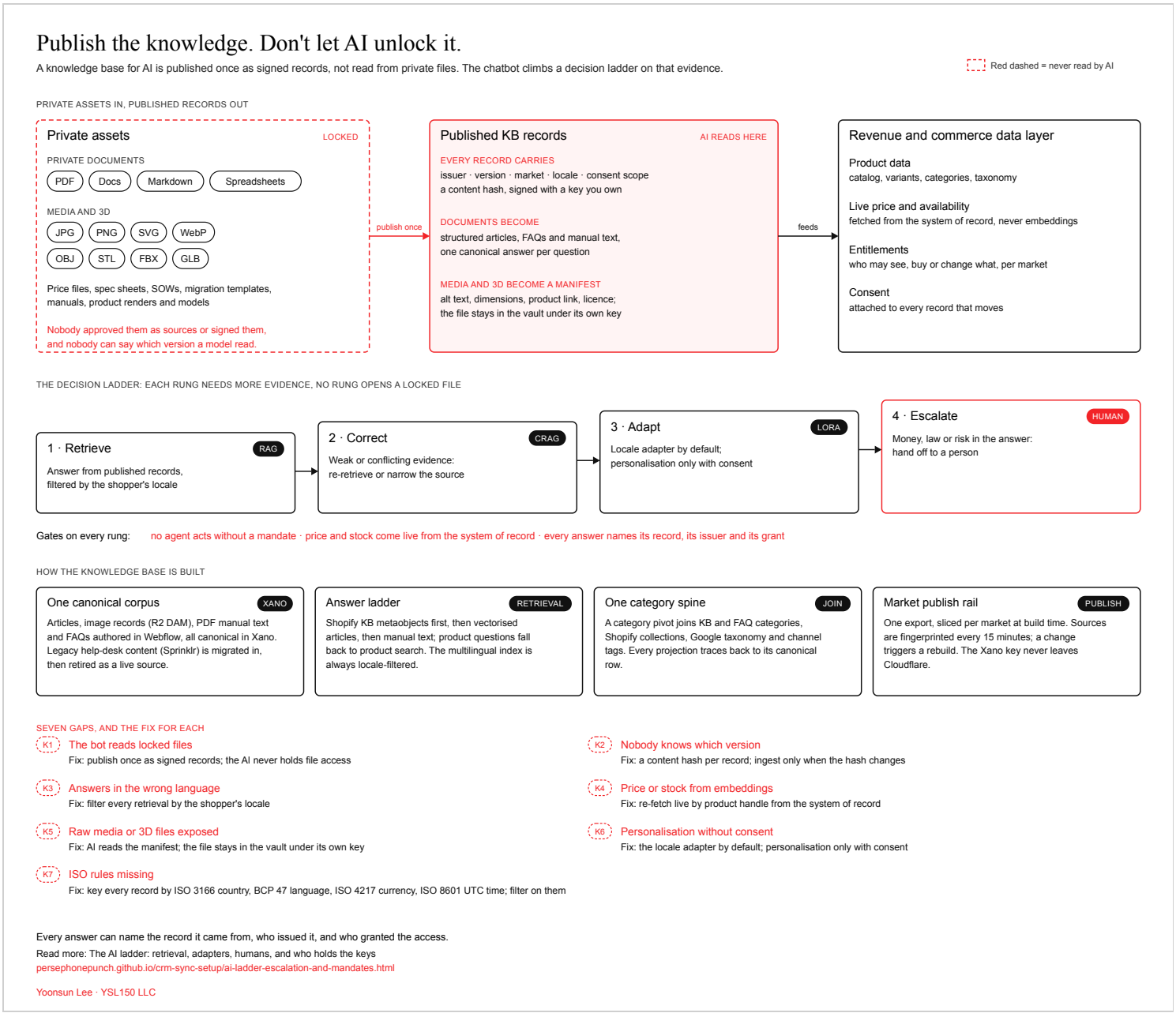
These are patterns to check in each app's terms and API, not claims about how any one app handles them.

5. Knowledge base: publish, don't unlock

Chatbots that answer from locked PDFs, docs, Markdown and spreadsheets, or from raw images and 3D files (JPG, PNG, SVG, WebP, OBJ, STL, FBX, GLB), read sources nobody approved, signed or versioned. A knowledge base for AI is **published once** as structured records carrying issuer, version, market, locale and consent scope, hashed and signed with a key you own. Media and 3D files become a manifest (alt text, dimensions, product link, licence) while the file stays in the vault under its own key.

The chatbot then climbs a decision ladder on that evidence: retrieval (RAG), corrective retrieval when evidence is weak (CRAG), a locale or personalisation

adapter (LoRA) only where consent allows, and a hand-off to a person when the answer carries money, law or risk. No rung opens a locked file, no agent acts without a mandate, and price and stock come live from the system of record, never from embeddings. More in The AI ladder.



Chatbot plugins: what they can't include

Most Shopify stores answer shoppers through a third-party app: helpdesk suites such as Gorgias, Zendesk and Sprinklr; SMS and conversational marketing such as Attentive and Postscript; on-site chat such as Tidio, Shopify Inbox and Intercom; and the AI agents each of them now adds. They run the bot

as a third-party script or service with the vendor's own storage, keys and certificates. That makes them a signing-keys-by-proxy problem of their own: whatever the shopper types lands in the vendor's transcripts, in the vendor's region, under the vendor's CA. Three things should stay out of them.

WHAT	TYPICAL CHATBOT APP (GORGIAS, ATTENTIVE, TIDIO, SPRINKLR AND OTHERS)	PUBLISHED-KB CHATBOT
Card data (PCI)	Must not be typed into chat: a transcript holding card numbers puts the vendor, and you, in PCI DSS scope, and masking depends on the vendor	Never accepted. Payment happens only in Shopify checkout
Personal data (PII) such as address, email, phone or ID numbers	Kept in the vendor's transcripts and region, outside your consent ledger, and missed when a deletion request runs	Not collected in chat. Identity comes from the signed-in account; consent is attached and deletion propagates
Checkout links	A generic product or cart link, without the shopper's market, consent state or grant	A URL hand-off to Shopify checkout with market and consent resolved; payment never in chat
Knowledge source	The vendor's copy of your help content, or private files uploaded to it	Published, signed records; helpdesk articles and macros (Gorgias, Sprinklr, Zendesk) are migrated into them, then retired as a live source
Keys and certificates	The vendor's CA and keys, inherited by proxy	Keys you own, verifiable offline

Check each vendor's terms and data-processing addendum for where transcripts are stored and how card numbers are masked; the point is that none of those choices are yours.

What each kind of app holds, and so what moves by proxy when you install it:

KIND OF APP	EXAMPLES	WHAT IT HOLDS ABOUT YOUR SHOPPERS	WHAT TO CHECK
Helpdesk and support chat	Gorgias, Zendesk, Sprinklr	Conversations, order lookups, macros and help articles, often with read and write access to orders and refunds	Which order and refund actions its AI agent can take without a person, and whether it writes around your gate
SMS and conversational marketing	Attentive, Postscript	Phone numbers, marketing consent, purchase and browsing signals, two-way message history	That marketing consent stays per channel and per market, and that opt-outs and deletions reach it on the event
On-site chat widgets	Tidio, Shopify Inbox, Intercom	Chat transcripts, visitor identifiers, and whatever the shopper types	That card data and personal data are blocked at input, and where transcripts are stored
AI agents added to any of the above	The vendors' own AI features	Training or retrieval over your help content and past conversations	Whether your content trains a shared model, and what happens to it when the subscription ends

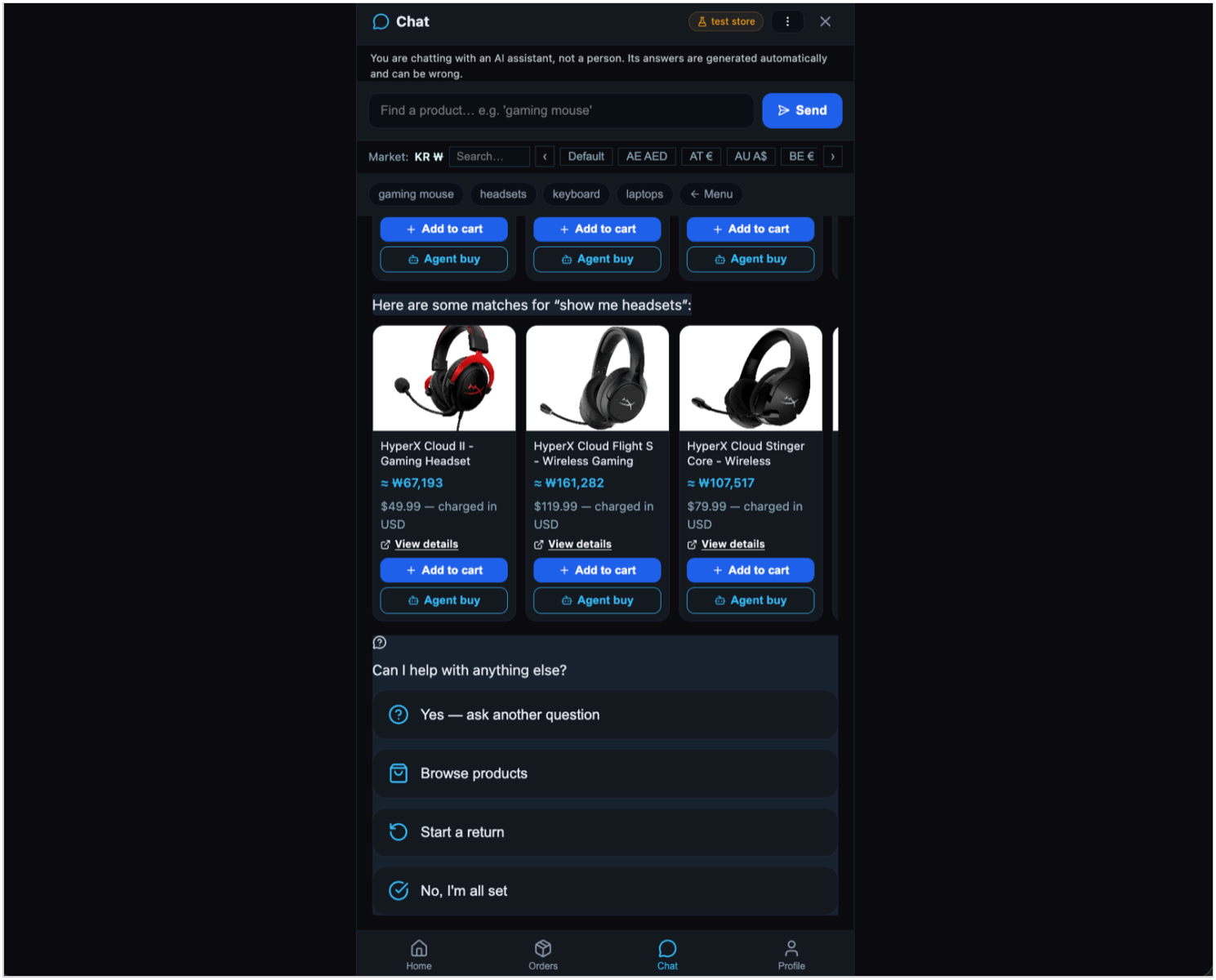
Chatbot build requirements

A headless Shopify proof of concept, the chatbot preview built on public product feeds and chatbot articles, shows the gap this section describes. It uses public data on purpose, to show the difference between being visible and being governed. Anything public can be viewed by AI, but that does not incorporate it into a knowledge base corpus with rules and permissions: no issuer, version, locale or consent scope, and no grant saying who may use it for what. Public articles an AI can read are still ungoverned until they are published as records. Its chatbot still answers from legacy Sprinklr articles that were never published through the knowledge base ladder, so its answers carry no

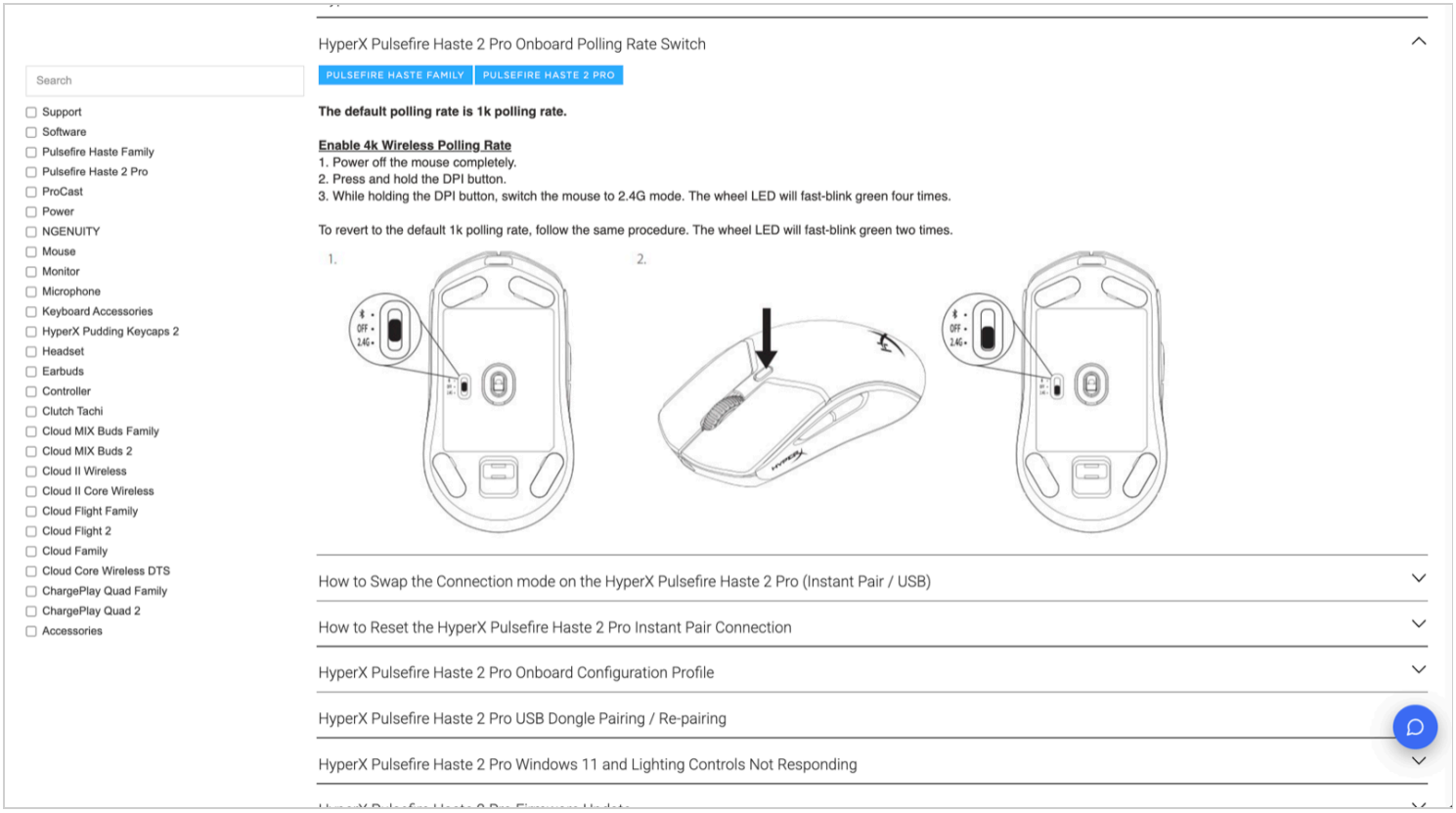
issuer, version, locale or consent scope, and nothing traces back to a canonical record. Closing that gap is the build:

Why an ungoverned ladder hallucinates. A chatbot or AI escalation ladder that retrieves from public docs without a governed corpus pulls *anything* that matches: an old article, another market's price, a machine translation, a page written for a different audience. With no guardrails, no global ISO rules and no personalised consent state, it has nothing to check an answer against, so it fills the gaps:

MISSING	WHAT THE LADDER DOES WITHOUT IT	GOVERNED CORPUS
Guardrails	Answers from any matching page, including retired or off-topic ones, and blends them	Retrieval reads published records only; weak evidence goes to CRAG or a person, never a guess
Global ISO rules	Mixes countries, languages, currencies and dates: a US price in a Korean answer, a stale date shown as current	Every record keyed by ISO 3166 country, BCP 47 language, ISO 4217 currency and ISO 8601 UTC time; retrieval filters on them
Personalised consent state	Personalises for everyone, or for no one, and can't tell which	Each answer checks the shopper's consent at the moment it's given; the locale adapter by default, personalisation only with consent
Provenance	Cites nothing, or cites a page that says something else	Each answer names its record, its issuer and its grant



The publishing side starts with a **curated knowledge base feed**: support articles tagged by product family, searchable and filterable, one answer per question, with step-by-step instructions and diagrams. Curation is the first rung. The next is publishing each entry as a signed record with its issuer, version, locale and consent scope, and pointing the chatbot at those records instead of the legacy helpdesk.



- ☐ **Migrate** each legacy helpdesk article (Sprinklr here; Gorgias, Zendesk or another helpdesk elsewhere) into a canonical record (one answer per question), then stop reading it live
- ☐ **Publish** every record with issuer, version, market, locale and consent scope, hashed and signed with a key you own
- ☐ **Re-index** retrieval from the published records only, filtered by locale
- ☐ **Hand off** checkout by URL to Shopify; take no card data and no personal data in chat
- ☐ **Trace** every answer to its record, its issuer and its grant
- ☐ **Retire** the helpdesk as a knowledge source once the published set covers its questions

LoRA audience training, linked to BigQuery and organic lookup

Two things are easy to confuse. **Audiences** decide *who* gets a personalised answer; **LoRA adapters** decide *how* the model writes for them: tone, vocabulary, format. BigQuery holds the audience, organic lookup supplies the evidence of

how each market actually asks, and the adapter is trained only on de-identified pairs with a training basis. Personal data never goes into model weights, because an erasure request can't reach inside a trained model.

```
BigQuery (consent-filtered, keyed by ISO country, language, currency, UTC time)
  ├─ audiences ─────→ GA4 export + Shopify orders → segments, pLTV (BigQuery ML)
  │                   → cohort labels, held below a minimum cohort size
  ├─ organic lookup ─→ Search Console bulk export: queries by country, language, device (aggreg
  │                   → KB gaps: questions searched but not answered → publish records
  │                   → evaluation set: real phrasing per market
  └─ training pairs ─→ redacted, de-identified question-and-answer pairs + cohort label
                      with training consent → clean room → LoRA adapter per market × cohort
                      → registry: dataset_hash, record_count, consent basis, retire_by
Chatbot: pickAdapter(locale, cohort, consent) → base model + adapter; facts from published recc
```

SOURCE IN BIGQUERY	USED FOR	NEVER USED FOR
GA4 export, with Consent Mode states	Cohort membership counts; which questions lead to conversion	Individual training rows
Shopify orders and pLTV scores	Cohort labels, under a pseudonymous ID	Weights that contain a person
Search Console bulk export (organic queries)	Knowledge base gaps; evaluation phrasing and vocabulary per market	Targeting a person: the data is aggregated by query
Redacted helpdesk question-and-answer pairs	Training pairs for tone and format	Facts: those stay in published records, so erasure still works

Rules that keep it governed:

- **Training consent is its own scope.** Marketing or personalisation consent is not consent to train; only records with a training basis enter the clean room.

- **Minimum cohort.** A cohort below the threshold gets no adapter and no label; it falls back to the locale adapter.
- **In-country training.** Korean data trains only on Korean-hosted systems; each market's rule from section 6 applies to training as well as storage.
- **Retire and retrain.** Every adapter carries `retire_by`; a withdrawal is honoured by retraining without the withdrawn records, not by editing weights.
- **Organic lookup feeds the corpus first.** A question people search for that the knowledge base can't answer becomes a published record before it becomes training data, so the adapter changes how the answer is written, never what it says.

Background: Google audiences and conversions with Vertex AI, and where LoRA fits and the clean room.

How `Vec<Tag>` organically feeds CRAG

Every published chunk carries a typed list of tags: market, locale, audience, consent, provenance, sensitivity and freshness. Nobody runs a tagging project. Events write the tags: an organic search that matches a doc adds an audience, signing at publish adds provenance, a consent withdrawal removes the consent tag. Corrective retrieval then uses them at every step. It pre-filters by market, locale and sensitivity, grades each chunk on signer, freshness and consent, then answers, re-queries one level wider, or declines and escalates with the reason. Whatever fails grading is logged with its tags and becomes the next round of audiences and published records. SAP and AEM user tags are free-text strings that nothing checks or expires. A Rust enum can't hold a typo or a lapsed consent, and an exhaustive `match` breaks the build until every grader handles a new rule.

How Vec<Tag> organically feeds CRAG

Events write typed tags onto each chunk; corrective retrieval filters, grades and corrects on them; what fails grading becomes the next round of tags.

1 · EVENTS WRITE THE TAGS (NO TAGGING PROJECT)

- Organic search in BigQuery matches a doc — `Audience(segment)`
- Doc signed at publish — `Provenance(signer)`
- Syndicated to a market — `Market(KR) · Locale(ko-KR)`
- Buyer grants or withdraws consent — `Consent(scope) added / removed`
- Feed or firmware update — `Freshness(now)`

writes

THE CHUNK OWNS ITS TAGS

```
enum Tag {
  Market(Country),          // ISO 3166
  Locale(LanguageTag),      // BCP 47
  Audience(SegmentId),      // BigQuery / organic lookup
  Consent(ConsentScope),    // what the buyer granted
  Provenance(SignerId),     // who signed the source
  Sensitivity(DataClass),   // Public | Pii | Pci
  Freshness(Timestamp),     // when it was last true
}
```

```
struct Chunk {
  text: String,
  embedding: Vec<f32>,
  tags: Vec<Tag>,
}
```

A typo, renamed campaign or expired consent can't pass as a valid tag: the compiler rejects anything outside the enum.

2 · CRAG USES THE TAGS AT EVERY STEP

1 Retrieve with a pre-filter

Vector query searches only chunks whose Market, Locale and Sensitivity match the request.

A KR shopper never retrieves a GB-only promotion.

2 Grade

Similarity score plus tag checks:

- Provenance: trusted signer?
- Freshness: within allowed age?
- Consent: scope covers this use?
- Sensitivity Pii | Pci: excluded.

3 Correct

Pass: answer from these chunks.

Ambiguous: re-query with tags relaxed one level (sibling locale).

Fail: decline and escalate to a person, with the tag reasons (below the 0.45 score bar).

4 Learn

Low-confidence and declined questions are logged with their tags.

That log feeds BigQuery audiences and de-identified LoRA training pairs: the next round of tags.

organic loop: failures become new tags

3 · WHY VEC<TAG>, AND WHY RUST

Ownership

The chunk owns its tags. Revoking consent removes Consent from the chunk itself: no stale copy left in a spreadsheet or a plugin cache.

Exhaustive match

Add a Tag variant and the build breaks until every grader handles it. A new rule can't be silently skipped: the hard-coded upsert failure.

Small and ordered

3 to 10 tags per chunk: a linear scan of a small Vec beats a hash map, and it serialises straight into vector-store metadata for the pre-filter.

vs SAP / AEM user tags

Free-text strings like "uk-promo-2024" set by a consultant. Nothing checks them, nothing expires them, and the chatbot retrieves them anyway.

4 · WHY RUST IN E-COMMERCE, CONSENT, SECURITY AND DATA LOAD

E-commerce

Price, stock and order state are typed values, not strings: a Money(amount, ISO 4217) can't be added to a different currency by accident.

Idempotency keys are a type, so a retried checkout or mandate never acts twice.

Consent

Consent is an owned value with a scope and an expiry. Withdrawal drops it, and every path that needs it fails to compile without a check.

source: business (a default) and source: platform (the buyer's choice) are different types.

Security

Memory safety removes the buffer overflows and use-after-free bugs behind about 70% of serious C and C++ CVEs (Microsoft, Chrome).

Keys are generated from a secure random source, zeroised on drop, and never copied into logs or caches.

Data load

Streams large feeds, catalogs and event logs with no garbage-collector pauses and predictable memory.

Parse once into types at the boundary; bad rows are rejected at load, not discovered by AI at answer time.

5 · EVERY THEME LOADS THROUGH THE CLOUDFLARE HELMET, REGARDLESS OF CMS PUBLISH

Any theme

Webflow → 11ty · Shopify · WordPress · AEM · Drupal · Joomla

The CMS publish owns presentation only

loads

Cloudflare helmet — vanilla HTML · CSS · JS · petite-vue · HTMX

1 Consent loads first
2 Origin paired to the tenant
3 Caps and mandate checked
4 Signs the release request

No React bundle, no theme API: any template carries it. No theme, plugin or module holds the gate secret.

signed

Xano release gate · source of truth

Consent at the system of record — Market, caps and residency
Fails closed; refusals ledgered — Kill switch refuses every release

Async Rust GET / POST / PATCH / DELETE from any stack: Tokio + reqwest or worker::Fetch. Vectorize, caches and feeds are derived copies.

6 · SHOPIFY PRODUCT GRAPHQL TO VERTEX AI AND CRAG, JOINED BY THE SHOPIFY GID

Shopify GraphQL

```
products {
  id handle title tags
  variants {
    sku barcode price } }
```

serde, at the boundary

Typed Product { gid, gtin, mpn, tags: Vec<Tag> }. A bad price or missing GTIN is rejected at load.

Xano · source of truth

Product record, consent and market rules. Every copy downstream is derived.

Vertex AI embeddings

Descriptions, specs, compatibility, FAQs, warranty. Never price or stock.

Vertex AI Vector Search

Tags become filter metadata: market, locale, sensitivity, consent.

CRAG answer

Pre-filter, grade, correct. Price and stock fetched live at answer time.

live GraphQL call for price and stock at answer time: a vector store holding last night's price is how a chatbot quotes a wrong price

Also: Shopify → BigQuery export feeds Vertex AI audiences and LoRA pairs, gated by consent. Merchant Center → Vertex AI Search for commerce shares the GTIN/MPN identities but does not grade against your consent and provenance tags.

7 · HEAP VS STACK, AND BIT LENGTH: WHERE KEY MATERIAL LIVES

	Stack	Heap
What lives there	Fixed size: u64, bool, [u8; 32], a struct's own fields	Size known at runtime: String text, Vec items, BigUint limbs
In serde	price_cents: u64 · available: bool · title: &'a str (borrowed, zero-copy)	title: String · tags: Vec<Tag> · variants: Vec<Variant> (owned)
Speed	Reserved and released as the function returns	Allocated, followed through a pointer, freed on drop
Secrets	Fixed-size key: one place in memory; zeroise on drop	A growing Vec can reallocate and leave old copies behind

Key or value	Bits	Bytes	Where it lives in Rust
Ed25519 private key	256	32	`[u8; 32] on the stack`
ECDSA P-256 / P-384	256 / 384	32 / 48	Fixed arrays on the stack
SHA-256 digest	256	32	`[u8; 32] on the stack`
RSA-2048 / RSA-4096	2048 / 4096	256 / 512	BigUint on the heap
Idempotency key, UUID	128	16	`u128 or [u8; 16]`

Heap and stack are where the bytes live, not how strictly they're typed: String is strongly typed and on the heap; u64 is strongly typed and on the stack. In Rust, both are checked by the compiler.

Why bit length matters for the eMudhra issue, and why it isn't enough

A longer key resists brute force; it doesn't help once the key itself is taken. The October 2026 intermediate was compromised, not cracked. What protects you: short-lived keys, HSM custody, fixed-size keys zeroised on drop, and a trust store you can distrust.

8 · KEY LENGTH IN TLS 1.2 VS TLS 1.3, AND WHAT THE EMUDHRA COMPROMISE DOES IN EACH

	TLS 1.2	TLS 1.3	Why it matters for the eMudhra issue
Key exchange	RSA key transport or (EC)DHE; static RSA still allowed	(EC)DHE only: forward secrecy is required	With RSA key transport, a stolen server key decrypts recorded past traffic; in 1.3 it can't
Exchange groups and sizes	Any DH size, including 1024-bit; curves as negotiated	Named groups only: X25519 (256), P-256, P-384, P-521, ffdhe2048 and up	Weak sizes can't be negotiated, so a downgrade can't shorten the key
Certificate key	RSA-2048 and up, ECDSA P-256 / P-384	The same, plus Ed25519; RSA signatures must use RSA-PSS	Certificate bit length doesn't change. The CA chain was compromised, so mis-issued certificates impersonate in both versions
Session (bulk) keys	AES-128 / 256 in CBC or GCM, 3DES (112-bit effective), ChaCha20	AEAD only: AES-128-GCM, AES-256-GCM, ChaCha20-Poly1305 (128 or 256-bit)	Short or legacy ciphers are gone

Hashes	SHA-1 and MD5 still appear in some signatures	SHA-256 / SHA-384, with HKDF for key derivation	No collision-prone hash in the handshake
Post-quantum	None	Hybrid X25519MLKEM768 key share	Traffic recorded today stays protected against a future quantum attacker

Events write the tags · the tags filter and grade retrieval · what fails grading flows back as the next round of tags.

Signing Keys by Proxy · CRM Sync KB

WHY RUST	WHAT IT PREVENTS
E-commerce	Money is typed with its ISO 4217 currency, and idempotency keys are a type, so a retried checkout or mandate never acts twice
Consent	Consent is an owned value with scope and expiry; a business default and a buyer's choice are different types
Security	Memory safety removes the overflow and use-after-free class; keys come from a secure random source and are zeroised on drop
Data load	Large feeds stream without garbage-collector pauses; bad rows are rejected at load, not discovered by AI at answer time

Where `Vec<Tag>` runs: Tokio/Axum or Workers

`Vec<Tag>` is plain Rust from the standard library, with no runtime attached. Tokio and Axum are one shell you can run it in; Cloudflare Workers is another. The tag model and the grader stay the same, and only the HTTP and async layer changes.

LAYER	SELF-HOSTED SERVER	CLOUDFLARE WORKERS	ROLE IN THE CRAG LOOP
Data model: `enum Tag`, `Vec<Tag>`	Standard Rust	Same code, compiled to WASM	What each chunk is and what it carries
Serialisation	serde	serde	Tags to JSON and vector-store metadata, and back
Async runtime	Tokio	The Workers runtime (no Tokio)	Runs many I/O tasks at once: webhooks, BigQuery, vector queries, and GET/POST/PATCH/DELETE to Xano
HTTP framework	Axum	`worker` crate (worker-rs), or TypeScript calling a Rust WASM module	Event webhooks that write tags; `/docs/ask` that runs retrieval
Vector store	Qdrant or pgvector	Vectorize	Pre-filter on tag metadata
Release	Cloudflare helmet, then the Xano release gate	The helmet is the Worker itself, then the Xano release gate	Every theme loads through the helmet; nothing leaves without a paired surface, a signed request and consent

Xano stays the source of truth, whatever stack compiles the code. Async Rust calls Xano's REST API with GET, POST, PATCH and DELETE, using the same tag model and the same idempotency key in every shell. On a server it runs on Tokio with `reqwest`. On Workers it uses `worker::Fetch`. In a CLI, a batch job or a WMS integration it's Tokio again. Each copy of a chunk, whether in Vectorize, a cache or a feed, is derived from the Xano record and reconciled against it. None of those copies becomes a second source of truth.

```
// Tokio + reqwest: write a tag change to Xano, the system of record
async fn withdraw_consent(x: &Xano, doc: DocId, scope: ConsentScope, key: IdemKey) -> Result<()
    let mut chunk: Chunk = x.get(&format!("/chunk/{doc}")).await?;             // GET
    chunk.tags.retain(|t| !matches!(t, Tag::Consent(s) if *s == scope));        // plain Vec
    x.patch(&format!("/chunk/{doc}"), &chunk, key).await?;                       // PATCH, idem
    x.post("/release/ledger", &LedgerEntry::withdrawn(doc, scope), key).await?; // POST, append
    Ok(())                                                                       // DELETE only
}
```

crm-sync runs on Workers, so the `.await` calls belong to the Workers runtime. The line that removes a withdrawn consent, `chunk.tags.retain(...)`, is ordinary `Vec` code in either shell.

6. Globalization: rules management by market

Each market sets whether data must stay in country, on what basis it may leave, and which Cloudflare and Xano options keep it where it belongs.

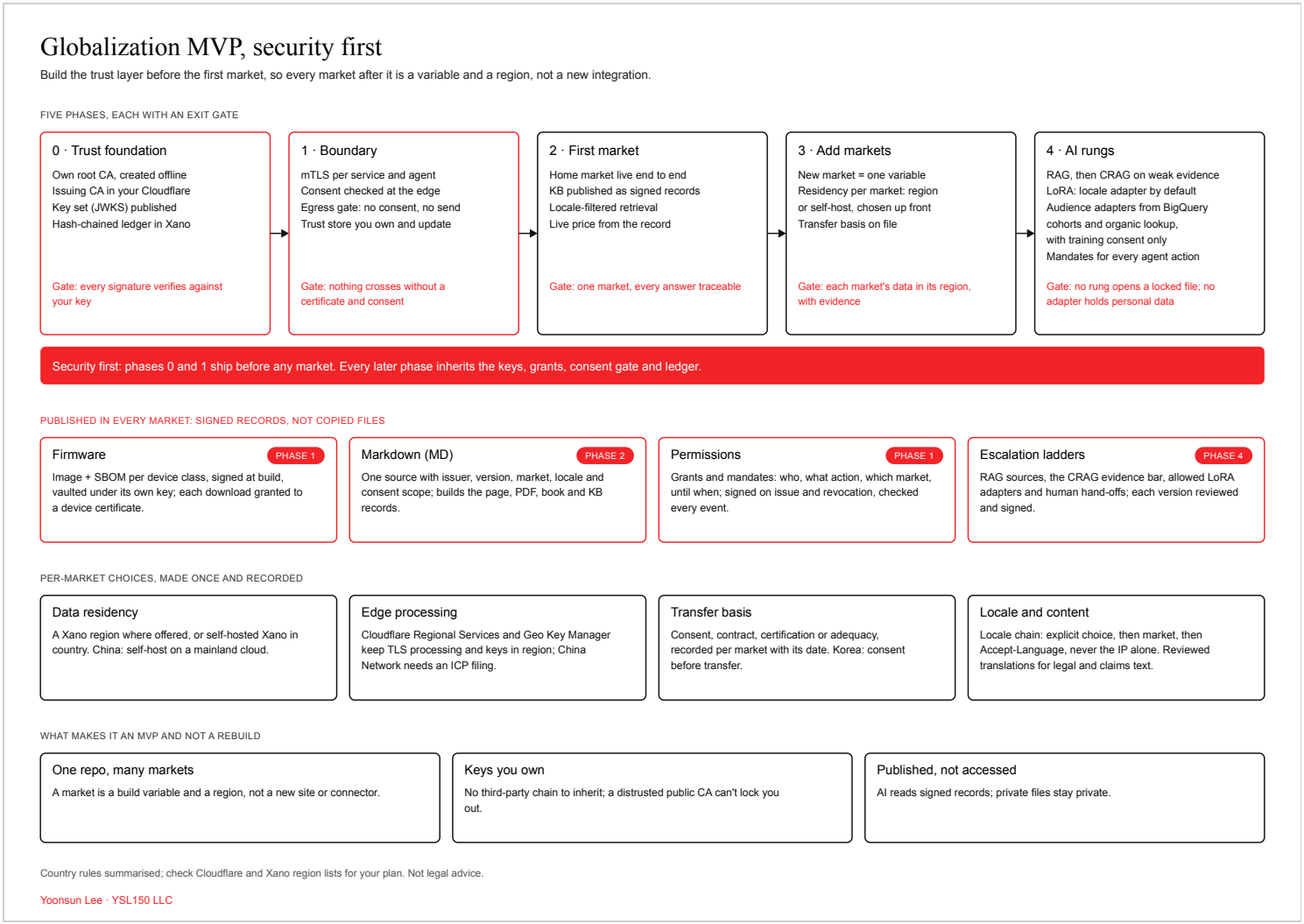
MARKET	LAW	DATA IN COUNTRY	CROSS-BORDER TRANSFER	CLOUDFLARE	XANO
China	PIPL, Data Security Law, Cybersecurity Law	Required for critical infrastructure operators and above volume thresholds; important data stays	Security assessment, standard contract or certification, unless an exemption applies	China Network via JD Cloud (Enterprise), needs an ICP filing; Geo Key Manager picks key regions	No mainland region: self-host on a mainland cloud, with its own TLS certificate from a CA you choose
Japan	APPI	Not required by law	Consent naming the country, or a recipient with equivalent safeguards; EU adequacy	Regional Services and Geo Key Manager keep TLS processing and keys in Japan	A Japan-region instance where offered, or self-host in Japan
Korea	PIPA (2023 amendment)	No general rule; sector rules for finance and public-sector cloud	Consent before transfer, or a certified or adequate recipient; disclose country and recipient	Regional Services and Geo Key Manager; check the Korea region	A Seoul-region instance, or self-host in Korea
Vietnam	Personal Data Protection Law (from 2026), Decree 53	Some services must store in Vietnam; foreign firms when ordered by the regulator	Transfer impact assessment filed with the Ministry of Public Security	No local residency product; keep personal data out of edge logs and caches	Self-host in Vietnam where localisation applies; otherwise a nearby region with the filing
Singapore	PDPA	Not required by law	Comparable protection: contract, binding corporate rules or certification	Regional Services and Geo Key Manager; check the region list	A Singapore-region instance where offered

MARKET	LAW	DATA IN COUNTRY	CROSS-BORDER TRANSFER	CLOUDFLARE	XANO
Taiwan	PDPA (2023 amendment)	Not general; sector rules	Allowed unless the regulator restricts a destination, e.g. mainland China for some sectors	Standard network; keep keys out of restricted regions	Nearest region or self-host; never route through a restricted destination
EU / Germany	GDPR, BDSG, TDDDG	Not required by law; often a contract term for public sector and health	Adequacy (including EU-US DPF, Japan, Korea, UK) or SCCs with a transfer assessment	Data Localization Suite: Regional Services, Customer Metadata Boundary, Geo Key Manager (EU)	An EU-region instance, or self-host in Germany
UK	UK GDPR, DPA 2018, Data (Use and Access) Act 2025	Not required by law	UK adequacy regulations, IDTA or Addendum; UK-US data bridge	Regional Services; check the UK region	A UK or EU-region instance, or self-host in the UK

Rules are summarised, not legal advice. Check Cloudflare's and Xano's current region lists for your plan; the AI cross-border requirements checklist goes deeper on transfer bases.

7. Globalization MVP, security first

Build the trust layer before the first market, so every market after it is a variable and a region, not a new integration.



PHASE	BUILD	EXIT GATE
0 · Trust foundation	Own root CA created offline; issuing CA in your Cloudflare account; key set (JWKS) published; hash-chained ledger in Xano	Every signature verifies against your key
1 · Boundary	mTLS per service and agent; consent checked at the edge; egress gate (no consent, no send); a trust store you own and update	Nothing crosses without a certificate and consent
2 · First market	Home market live end to end; knowledge base published as signed records; locale-filtered retrieval; live price from the system of record	One market, every answer traceable
3 · Add markets	A new market is one build variable; residency per market (region or self-host) chosen up front; transfer basis on file	Each market's data in its region, with evidence
4 · AI rungs	RAG, then CRAG on weak evidence; LoRA with the locale adapter by default, and audience adapters trained from BigQuery cohorts and organic lookup only with training consent; mandates for every agent action	No rung opens a locked file; no adapter holds personal data

Phases 0 and 1 ship before any market. Every later phase inherits the keys, grants, consent gate and ledger, which is what keeps it an MVP rather than a rebuild per market.

Publishing requirements for the MVP

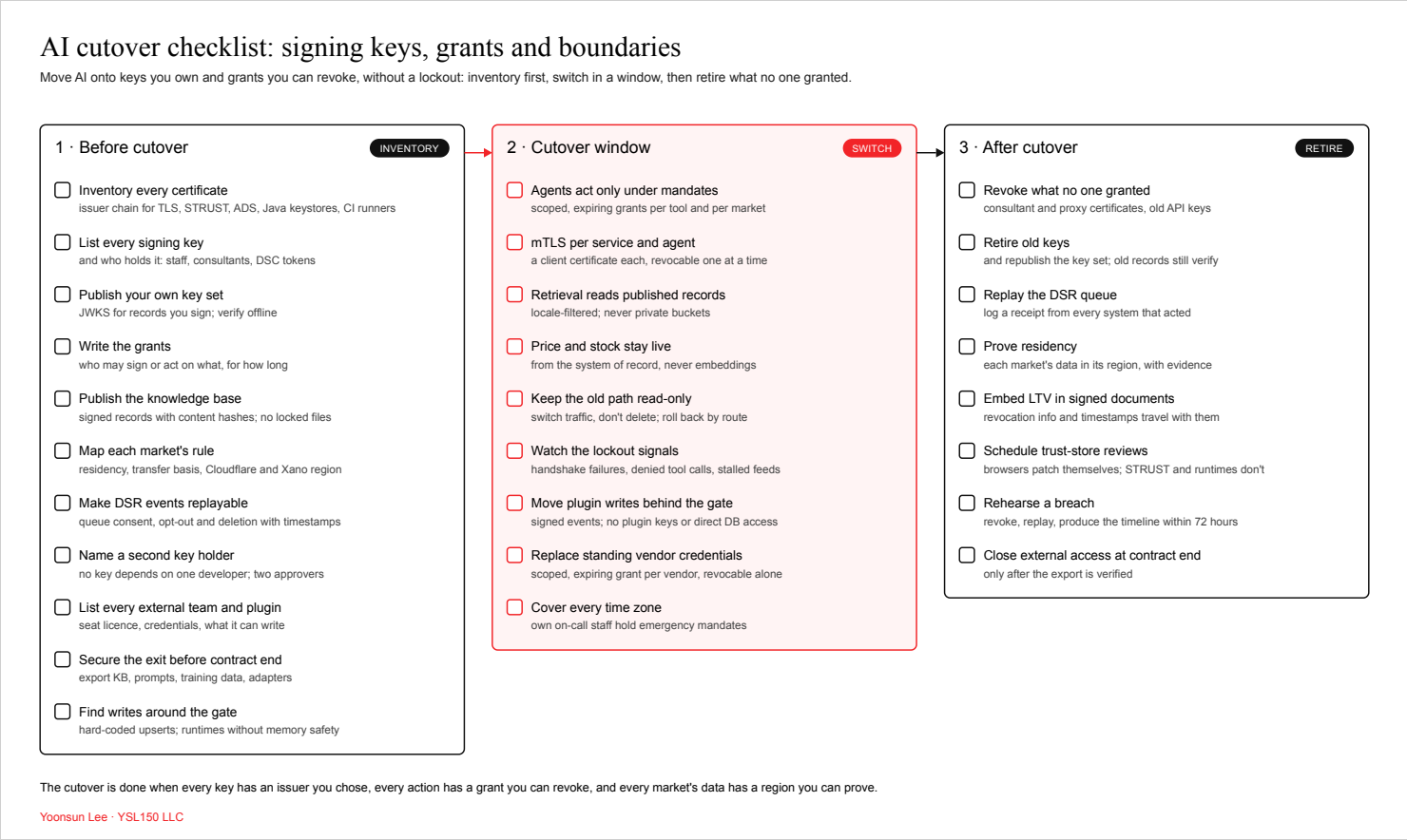
Four things are published in every market, and each is a signed record, not a file someone copies into a system:

PUBLISHED	SOURCE AND FORMAT	SIGNED AND CHECKED	PHASE AND GATE
Firmware	The image plus its SBOM, per device class and version	Signed at build; vaulted under its own key; each download granted to a device certificate and recorded in the ledger; devices verify before install	Phase 1: no device runs an image it can't verify
Markdown (MD)	Markdown with frontmatter carrying issuer, version, market, locale and consent scope: the one source for the web page, the PDF, the book and the knowledge base records	Content-hashed and signed at publish; every surface traces back to the same commit	Phase 2: every answer and page traces to one source
Permissions	Grants and mandates as records: who or which agent, which action, which market, until when	Signed when issued, and again when revoked; checked on every event, never cached past expiry	Phase 1: nothing acts without a current grant
Escalation ladders	Per market: which sources RAG may read, the evidence bar that sends a question to CRAG, which LoRA adapters are allowed and under what consent, and which answers go to a person (money, law, risk)	Each ladder version reviewed (a QA attestation) and signed before it goes live; the version in force is recorded with every answer	Phase 4: no rung changes without a signed review

A new market reuses all four: its firmware channel, its Markdown slice, its grants and its ladder are configuration on the same rails, not a new integration.

8. AI cutover checklist

Move AI onto keys you own and grants you can revoke, without a lockout: inventory first, switch in a window, then retire what no one granted.



Before cutover: inventory

- ☐ **Inventory every certificate**, with its issuer chain: TLS, `STRUST`, ADS, Java keystores, CI runners
- ☐ **List every signing key** and who holds it: staff, consultants, DSC tokens
- ☐ **Publish your own key set** (JWKS) for the records you sign, so they verify offline
- ☐ **Write the grants**: who may sign or act on what, for how long
- ☐ **Publish the knowledge base** as signed records with content hashes; no locked files

- [ ] **Map each market's rule**: residency, transfer basis, Cloudflare and Xano region
- [ ] **Make DSR events replayable**: queue consent, opt-out and deletion with timestamps
- [ ] **Name a second key holder**: no key or rotation depends on one developer; two approvers and a written runbook
- [ ] **List every external team and plugin with access**: its seat licence, its credentials and what it can write
- [ ] **Secure the exit before the contract ends**: export the knowledge base, prompts, labelled data and adapters, and confirm the export opens
- [ ] **Find writes around the gate**: plugins with hard-coded upserts, or running without memory safety

Cutover window: switch

- [ ] **Agents act only under mandates**: scoped, expiring grants per tool and per market
- [ ] **mTLS per service and agent**: a client certificate each, revocable one at a time
- [ ] **Retrieval reads published records** only, locale-filtered; never private buckets
- [ ] **Price and stock stay live** from the system of record, never embeddings
- [ ] **Keep the old path read-only**: switch traffic, don't delete; roll back by route
- [ ] **Watch the lockout signals**: handshake failures, denied tool calls, stalled feeds
- [ ] **Move plugin writes behind the gate**: every write a signed event; no plugin holds signing keys or direct database access
- [ ] **Replace standing vendor credentials**: a scoped, expiring grant and certificate per vendor, revocable alone

- [ ] **Cover every time zone**: your own on-call staff hold pre-approved emergency mandates

After cutover: retire

- [ ] **Revoke what no one granted**: consultant and proxy certificates, old API keys
- [ ] **Retire old keys** and republish the key set; old records still verify
- [ ] **Replay the DSR queue** and log a receipt from every system that acted
- [ ] **Prove residency**: each market's data in its region, with evidence
- [ ] **Embed LTV** in signed documents, so revocation info and timestamps travel with them
- [ ] **Schedule trust-store reviews**: browsers patch themselves; `STRUST` and runtimes don't
- [ ] **Rehearse a breach**: revoke a key, replay the queue and produce the timeline within GDPR's 72 hours
- [ ] **Close external access at contract end**: only after the export is verified

Contract clauses to ask for

- **Disclose every provider**: every trust service provider and CA used for your signers or systems.
- **Certificates issued to the client**: certificates installed in your systems name you, not the consultancy.
- **Embed LTV**: revocation info and timestamps embedded, so documents stay verifiable after the relationship ends.

The cutover is done when every key has an issuer you chose, every action has a grant you can revoke, and every market's data has a region you can prove.

Related

- Org Trust Layer: your own CA, fewer plugins
- The AI ladder: retrieval, adapters, humans, and who holds the keys
- Entitlement security
- Hydrogen or bespoke stack
- AI cross-border requirements checklist
- Chatbot preview and curated knowledge base feed

Patterns, not claims about any firm. eMudhra incident details per The Hacker News and Help Net Security, October 2026. Country rules summarised; not legal advice.
